Problem	**Topics**	**Difficulty**	**Solver**	**Notes**
A				
B				
C				
D				
E				
F				
G				
H				
I				
J				
K				
L				
M				
N				
O				
P				
Q				

ICPC Team Reference Material

Contents

1 Setup 2
- 1.1 Troubleshoot . . . 2
- 1.2 Vimrc . . . 3
- 1.3 C++ template . . . 4
- 1.4 Capslock as escape && copy template . . . 4
- 1.5 Compilation . . . 4

2 Data Structure 5
- 2.1 Segmented Tree . . . 5
- 2.2 Segmented Tree 2 . . . 6

3 Graph algorithms 8
- 3.1 Dfs . . . 8
- 3.2 Bfs . . . 9
- 3.3 Dijkstra's . . . 10
- 3.4 Toplogical Sort . . . 10
- 3.5 Lowest Common Ancestor . . . 11

4 Mathematics 12
- 4.1 Count Inversions . . . 12
- 4.2 Ncr . . . 12
- 4.3 Fastpower . . . 13
- 4.4 Simple Sieve . . . 13
- 4.5 Prime Factorization (Log N) . . . 14
- 4.6 Calculate All Divisors . . . 14
- 4.7 Prime Factorization . . . 15
- 4.8 Fast matrix mult and pow . . . 15
- 4.9 Josephus . . . 15
- 4.10 Pascal . . . 16
- 4.11 Modular Inverse . . . 16

5 Miscellaneous 16
- 5.1 Gcd & Lcm . . . 16
- 5.2 Debugging tools . . . 17
- 5.3 Pseudo random number generator . . . 17
- 5.4 Increase the stack memory limit . . . 17
- 5.5 Stress test . . . 18
- 5.6 Extra stuff . . . 18
- 5.7 Knapsack dp . . . 21

1 Setup

1.1 Troubleshoot

```
General:
Write down most of your thoughts, even if you're not sure whether they're useful.
Give your variables (and files) meaningful names
Stay organized and don't leave papers all over the place!
You should know what your code is doing..

Pre-submit:
Write a few simple test cases if the sample is not enough.
Are time limits close? If so, generate max cases.
Is the memory usage fine?
Could anything overflow?
Remove debug output
Make sure to submit the right file

Wrong answer:
Print your solution! Print debug output as well.
Read the full problem statement again
Have you understood the problem correctly?
Are you sure your algorithm works?
Try writing a slow (but correct) solution
Can your algorithm handle the whole range of input?
Did you consider corner cases (e.g., n=1)?
Is your output format correct? (including whitespace)
Are you clearing all data structures between test cases?
```

```
Any uninitialized variables?
Any undefined behavior (array out of bounds)?
Any overflows or NaNs (or shifting long long by >=64 bits)?
Confusing N and M, i and j, etc.?
Confusing ++i and i++?
Return vs continue vs break?
Are you sure the STL functions you use work as you think?
Add some assertions, maybe resubmit.
Create some test cases to run your algorithm on
Go through the algorithm for a simple case
Go through this list again
Explain your algorithm to a teammate
Ask the teammate to look at your code
Go for a small walk, e.g., to the toilet.
Rewrite your solution from the start or let a teammate do it

Geometry:
Work with ints if possible
Correctly account for numbers close to (but not) zero
 - For functions like acos, make sure the absolute value of the input is not (slightly) greater than one.
Correctly deal with vertices that are collinear, concyclic, coplanar (in 3D), etc.
Subtracting a point from every other (but not itself)?

Runtime error:
Have you tested all corner cases locally?
Any uninitialized variables?
Are you reading or writing outside the range of any vector?
Any assertions that might fail?
Any possible division by 0? (mod 0, for example)
Any possible infinite recursion?
Invalidated pointers or iterators?
Are you using too much memory?
Debug with resubmits (e.g., remapped signals, see Various).

Time limit exceeded:
Do you have any possible infinite loops?
What's your complexity? Large TL does not mean that something simple (like NlogN) isn't intended.
Are you copying a lot of unnecessary data? (Use references)
Avoid vector, map (Use arrays/unordered_map)
How big is the input and output? (Consider FastIO)
What do your teammates think about your algorithm?
Calling count() on multiset?

Memory limit exceeded:
What is the max amount of memory your algorithm should need?
Are you clearing all data structures between test cases?
If using pointers, try BumpAllocator
```

1.2 Vimrc

```
# init.vim
let mapleader = " "
syntax on
filetype plugin on
colorscheme  desert
set nocompatible
set autoread
set foldmethod=marker
set autoindent
set clipboard+=unnamedplus
set number relativenumber
```

```
set cursorline
set shiftwidth=2 softtabstop=2 expandtab
set undofile
set undodir=~/.vim/undo
set undolevels=1000
set undoreload=10000
map <leader>r :w! <CR> :!./compile "%:p:r"<CR>:!"%:p:r"<CR>
map <leader>x <cmd> %y+ <CR>
```

1.3 C++ template

```
#include <bits/stdc++.h>

#define endl '\n'
#define int long long
#define ld  long double
#define all(a) (a).begin(), (a).end()
#define sz(a) (int)(a).size()
#define pb push_back
#define F first
#define S second
#define vi vector<int>

using namespace std;

void file() {
  freopen("input.in", "r", stdin);
  freopen("output.out", "w", stdout);
}

void Solve() {
}

int32_t main() {
  ios_base::sync_with_stdio(false);
  cin.tie(nullptr);
  // file();

  int t = 1;
  // cin >> t;
  for(int i = 1; i <= t; ++i) {
    Solve();
  }
}
```

1.4 Capslock as escape && copy template

```
setxkbmap -layout us
xmodmap -e 'clear Lock'
xmodmap -e 'keycode 66 = Escape'

! copy template
printf "%s\n" {b..q} | xargs -I {} cp a.cpp {}.cpp

! Add custom scripts to PATH put in bashprofile
PATH="$PATH:/home/nero"
export PATH
```

1.5 Compilation

```
#!/bin/bash
# put this file in .local/bin or add its dir to the PATH variable
compile() {
```

```
  g++ -Wall -Wextra -Wshadow -Ofast -std=c++17 -pedantic -Wformat=2 -Wconversion -Wlogical-op -Wshift-overflow=2 -Wduplicated-cond -Wfloat-equal -fno-sanitize-recover -fstack-protector -fsanitize=address,undefined -fmax-errors=2 -o "$1"{,.cpp}
}
compile "$1"

# Compile 2.0

#!/bin/bash
while true; do
    clear
    g++ a.cpp
    ./a.out < input_a

    sleep 3s
    vim a.cpp
done
```

2 Data Structure

2.1 Segmented Tree

```
class SegmentTree {
public:
  SegmentTree(int n) {
    size = 1;
    while (size < n) size <<= 1;
    sums.assign(2 * size, 0LL);
    mins.assign(2 * size, LLONG_MAX);
    maxs.assign(2 * size, LLONG_MIN);
  }

  void build(const vi &a) {
    build(a, 0, 0, size);
  }

  void update(int i, int v) {
    update(i, v, 0, 0, size);
  }

  ll sumSeg(int l, int r) {
    return sumSeg(l, r, 0, 0, size);
  }

  ll minSeg(int l, int r) {
    return minSeg(l, r, 0, 0, size);
  }

  ll maxSeg(int l, int r) {
    return maxSeg(l, r, 0, 0, size);
  }

private:
  int size;
  vector<ll> sums, mins, maxs;

  void build(const vi &a, int x, int lx, int rx) {
    if (rx - lx == 1) {
      if (lx < (int)a.size()) {
        sums[x] = a[lx];
        mins[x] = a[lx];
        maxs[x] = a[lx];
      }
      return;
```

```
    }
    int mid = (lx + rx) / 2;
    build(a, 2 * x + 1, lx, mid);
    build(a, 2 * x + 2, mid, rx);
    sums[x] = sums[2 * x + 1] + sums[2 * x + 2];
    mins[x] = min(mins[2 * x + 1], mins[2 * x + 2]);
    maxs[x] = max(maxs[2 * x + 1], maxs[2 * x + 2]);
  }

  void update(int i, int v, int x, int lx, int rx) {
    if (rx - lx == 1) {
      sums[x] = v;
      mins[x] = v;
      maxs[x] = v;
      return;
    }
    int mid = (lx + rx) / 2;
    if (i < mid) {
      update(i, v, 2 * x + 1, lx, mid);
    } else {
      update(i, v, 2 * x + 2, mid, rx);
    }
    sums[x] = sums[2 * x + 1] + sums[2 * x + 2];
    mins[x] = min(mins[2 * x + 1], mins[2 * x + 2]);
    maxs[x] = max(maxs[2 * x + 1], maxs[2 * x + 2]);
  }

  ll sumSeg(int l, int r, int x, int lx, int rx) {
    if (lx >= r || l >= rx) return 0;
    if (lx >= l && rx <= r) return sums[x];
    int mid = (lx + rx) / 2;
    ll left = sumSeg(l, r, 2 * x + 1, lx, mid);
    ll right = sumSeg(l, r, 2 * x + 2, mid, rx);
    return left + right;
  }

  ll minSeg(int l, int r, int x, int lx, int rx) {
    if (lx >= r || l >= rx) return LLONG_MAX;
    if (lx >= l && rx <= r) return mins[x];
    int mid = (lx + rx) / 2;
    ll left = minSeg(l, r, 2 * x + 1, lx, mid);
    ll right = minSeg(l, r, 2 * x + 2, mid, rx);
    return min(left, right);
  }

  ll maxSeg(int l, int r, int x, int lx, int rx) {
    if (lx >= r || l >= rx) return LLONG_MIN;
    if (lx >= l && rx <= r) return maxs[x];
    int mid = (lx + rx) / 2;
    ll left = maxSeg(l, r, 2 * x + 1, lx, mid);
    ll right = maxSeg(l, r, 2 * x + 2, mid, rx);
    return max(left, right);
  }
};
```

2.2 Segmented Tree 2

```

# STL policy container (oset, omap)
    #include<ext/pb_ds/assoc_container.hpp>
    #include<ext/pb_ds/tree_policy.hpp>
    using namespace __gnu_pbds;
    template<typename T>
    using ordered_set = tree<T, null_type, std::less<T>, rb_tree_tag,
```

```
        tree_order_statistics_node_update>;


# optimized segment tree with basic operations
    template<typename T = long long>
    struct Sum {
        T value;
        Sum(T value = 0) : value(value) {}
        Sum &operator+=(const Sum &other) { return value += other.value, *this; }
        Sum operator+(const Sum &other) const { return value + other.value; }
    };

    template<typename T = long long>
    struct Max {
        T value;
        Max(T value = numeric_limits<T>::min() / 2) : value(value) {}
        Max &operator+=(const Max &other) { return value = max(value, other.value), *this; }
        Max operator+(const Max &other) const { return Max(max(value, other.value)); }
    };

    template<typename T = long long>
    struct Min {
        T value;
        Min(T value = numeric_limits<T>::max() / 2) : value(value) {}
        Min &operator+=(const Min &other) { return value = min(value, other.value), *this; }
        Min operator+(const Min &other) const { return Min(min(value, other.value)); }
    };

    // source: https://codeforces.com/blog/entry/18051
    template<typename T>
    struct Segtree {
        int n;
        vector<T> tree;

        Segtree() = default;
        Segtree(int n) : n(n) {
            tree.resize(n * 2);
        }

        void build() {
            for (int i = n - 1; i > 0; --i)
                tree[i] = tree[i << 1] + tree[i << 1 | 1];
        }

        void update(int i, T val) {
            for (tree[i += n] = val; i > 1; i >>= 1)
                tree[i >> 1] = tree[i] + tree[i ^ 1];
        }

        auto query(int l, int r) {
            T res;
            for (l += n, r += n + 1; l < r; l >>= 1, r >>= 1) {
                if (l & 1) res += tree[l++];
                if (r & 1) res += tree[--r];
            }
            return res.value;
        }
    };
```

```
// Find the element nearest to x:

```

```
auto it = s.lower_bound(x);
if (it == s.begin()) {
    cout << *it << "\n";
} else if (it == s.end()) {
    it--;
    cout << *it << "\n";
} else {
    int a = *it; it--;
    int b = *it;
    if (x-b < a-x) cout << b << "\n";
    else cout << a << "\n";
}
```

3 Graph algorithms

3.1 Dfs

```
vector<vector<int>> graph;
vector<bool> visited;

void dfs(int start) {
    stack<int> stack;
    stack.push(start);

    while (!stack.empty()) {
        int node = stack.top();
        stack.pop();

        if (!visited[node]) {
            cout << char(node + 'A') << ' ';
            visited[node] = true;
        }

        for (auto it = graph[node].rbegin(); it != graph[node].rend();
           ++it) {
            if (!visited[*it]) {
                stack.push(*it);
            }
        }
    }
}

int n; // number of vertices
vector<vector<int>> adj; // adjacency list of graph
vector<bool> visited;
vector<int> ans;

void dfs(int node) {
    visited[node] = true;
    cout << node << ' ';

    for (int neighbor : graph[node]) {
        if (!visited[neighbor]) {
            dfs(neighbor);
        }
    }

    // ans.push_back(node);
}

void topological_sort() {
    visited.assign(n, false);
    ans.clear();
    for (int i = 0; i < n; ++i) {
        if (!visited[i]) {
```

```
            dfs(i);
        }
    }
    reverse(ans.begin(), ans.end());
}
```

3.2 Bfs

```
vector<vector<int>> adj; // adjacency list
vector<int> vis(n+1, 0);
vector<int> p(n+1, -1); // parent of node

void bfs(int source) {
    queue<int> q;
    q.push(s);
    p[s]=-1; // root

    while(!q.empty()) {
        int g = q.front();
        cout << g << endl;
        vis[g]=1;
        q.pop();
        for(int v : adj[g]) {
            if(!vis[v]) {
                vis[v]=1;
                q.push(v);
                p[v]=g;
                cout << v << endl;
            }
        }
    }

    // to find the shortest path using BFS
    vector<int> path;
    int target = 5;   // targeted node
    for(int v = target; v != -1; v = p[v]) {
        path.push_back(v);
    }
    reverse(path.begin(), path.end());
    for (int i = 0; i < path.size(); i++) {
        cout << path[i];
        if(i!=path.size()-1) {
            cout << " --> ";
        }
    }

    // to count connected components using BFS
    int cnt = 0;
    vis = vector<int> (n+1, 0);
    for (int i = 1; i <= n; i++) {
        if(!vis[i]) {
            q.push(i);
            vis[i]=1;
            while(!q.empty()) {
                int g = q.front();
                q.pop();
                vis[g]=1;
                for(int v : adj[g]) {
                    if(!vis[v]) {
                        vis[v]=1;
                        q.push(v);
                        p[v]=g;
                    }
                }
            }
```

```
            cnt++;
        }
    }
    cout << "cnt= " << cnt << endl;
}
```

3.3 Dijkstra's

```
int n;
vector<vector<pair<int, int> > > adj;
void addEdge(int u, int v, int w) {
  adj[u].emplace_back(v, w);
  adj[v].emplace_back(u, w);
}
vi visited;
vi dijkstra(int x) {
  vi distance(adj.size(), INT_MAX);
  priority_queue<pair<int, int>, vector<pair<int, int>>, greater<> > q;
  q.emplace(0, x);
  distance[x] = 0;
  q.emplace(0, x);
  while (!q.empty()) {
    int a = q.top().second;

    q.pop();
    if (visited[a]) continue;
    visited[a] = true;
    for (auto u : adj[a]) {
      int b = u.first, w = u.second;
      if (distance[a] + w < distance[b]) {
        distance[b] = distance[a] + w;
        q.emplace(distance[b], b);
        debug(distance);
      }
    }
  }
  return distance;
}
void printSolution(vi dist) {
  for (int i = 0; i < n; i++) cout << i << " \t\t-->\t\t" << dist[i] <<
     endl;
}
```

3.4 Toplogical Sort

```
int n; // number of vertices
vector<vector<int>> adj; // adjacency list of graph
vector<bool> visited;
vector<int> ans;

void dfs(int v) {
    visited[v] = true;
    for (int u : adj[v]) {
        if (!visited[u])
            dfs(u);
    }
    ans.push_back(v);
}

void topological_sort() {
    visited.assign(n, false);
    ans.clear();
    for (int i = 0; i < n; ++i) {
```

```
        if (!visited[i]) {
            dfs(i);
        }
    }
    reverse(ans.begin(), ans.end());
}
```

3.5 Lowest Common Ancestor

```
// O(NlogN) for preprocessing, and O(logN) for each LCA query

int n, l, timer; // only assign n equal to # of vertices in main
vvi adj, up; // also Adjacency list :)
vi tin, tout;

void dfs(int v, int p){
    tin[v] = ++timer;
    up[v][0] = p;
    for (int i = 1; i <= l; ++i)
        up[v][i] = up[up[v][i - 1]][i - 1];

    for (auto u : adj[v]) if (u != p) dfs(u, v);
    tout[v] = ++timer;
}

bool is_ancetor(int u, int v){
    return tin[u] <= tin[v] && tout[u] >= tout[v];
}

int lca(int u, int v){
    if (is_ancetor(u, v)) return u;
    if (is_ancetor(v, u)) return v;

    for (int i = l; i >= 0; --i)
        if (!is_ancetor(up[u][i], v))
            u = up[u][i];

    return up[u][0];
}

void preprocess(int root) {
    tin.resize(n), tout.resize(n);
    timer = 0;
    l = ceil(log2(n));
    up.assign(n, vi(l + 1));
    dfs(root, root);
}
```

```
bool dfs(int v) {
    // if this function returned true, count++

    visited[v] = true;

    for (int i : graph[node]) {
        if (!visited[i]) {
            if (dfs(i)) {
                return true;
            }
        } else if (i != parent) {
            return true;
        }
    }
    return false;
}
```

4 Mathematics

4.1 Count Inversions

```
int count_inversions_permutation(vector<int>& A) {
    int cnt = 0;
    for (int i = 0; i < A.size(); i++) {
        cnt += abs((i+1) - A[i]);
    }
    return cnt/2;
}

int count_inversions(vector<int>& A) {
    int cnt = 0;
    vector<int> B;
    B.push_back(A[0]);
    for (int i = 1; i < A.size(); i++) {
        auto it = lower_bound(B.begin(), B.end(), A[i]);
        B.insert(it, A[i]);

        auto it_last = upper_bound(B.begin(), B.end(), A[i]);
        int idx_last = it_last - B.begin();

        cnt += B.size() - idx_last;
    }
    return cnt;
}
```

4.2 Ncr

```
#include <bits/stdc++.h>
#define int long long
using namespace std;

template<class T>
using rpq = priority_queue<T, vector<T>, greater<T>>;

template<int32_t mod>
struct mint {
    using Z = mint;
    int32_t x;
    mint(int32_t x = 0) : x(norm(x)) {}
    mint(long long x) : x(norm(x % mod)) {}
    inline int32_t norm(int32_t x) const {
        return x >= mod ? x - mod : (x < 0 ? x + mod : x);
    }
    Z power(long long b) const {
        Z res = 1, a = x;
        for (; b; b >>= 1, a *= a)
            if (b & 1) res *= a;
        return res;
    }
    Z inv() const { return assert(x != 0), power(mod - 2); }
    Z operator-() const { return -x; }
    Z &operator*=(const Z &r) { return *this = (long long) x * r.x; }
    Z &operator+=(const Z &r) { return *this = x + r.x; }
    Z &operator-=(const Z &r) { return *this = x - r.x; }
    Z &operator/=(const Z &r) { return *this *= r.inv(); }
    friend Z operator*(const Z &l, const Z &r) { return Z(l) *= r; }
    friend Z operator+(const Z &l, const Z &r) { return Z(l) += r; }
    friend Z operator-(const Z &l, const Z &r) { return Z(l) -= r; }
    friend Z operator/(const Z &l, const Z &r) { return Z(l) /= r; }
    friend ostream &operator<<(ostream &os, const Z &a) { return os << a
        .x; }
```

```
    friend istream &operator>>(istream &is, Z &a) {
        long long y = 0;
        return is >> y, a = y, is;
    }
};

// constexpr int MOD = 998244353;
constexpr int MOD = 1000000007;
using Z = mint<MOD>;

vector<Z> fact = {1};
vector<Z> fact_inv = {1};

void build_fact(int n = 1e6) {
    while ((int) fact.size() < n + 1)
        fact.push_back(fact.back() * (int) fact.size());
    fact_inv.resize(fact.size());
    fact_inv.back() = fact.back().inv();
    for (int j = fact_inv.size() - 2; fact_inv[j].x == 0; j--)
        fact_inv[j] = fact_inv[j + 1] * (j + 1);
}

Z ncr(int n, int r) {
    if (r > n || r < 0) return 0;
    if ((int) fact.size() < n + 1) build_fact(n);
    return fact[n] * fact_inv[r] * fact_inv[n - r];
}

Z npr(int n, int r) {
    if (r > n || r < 0) return 0;
    if ((int)fact.size() < n + 1) build_fact(n);
    return fact[n] * fact_inv[n - r];
}
```

4.3 Fastpower

```
int fast_power(int a, int b) {
    int res = 1;
    while (b) {
        if (b & 1) res = res * a % mod;
        a = a * a % mod;
        b >>= 1;
    }
    return res;
}
```

4.4 Simple Sieve

```
const int NMAX = 1000000;
bitset<NMAX / 2> bits;

void precalcseive() {
  bits.set();
  for (int i = 3; i / 2 < bits.size(); i = 2 * bits._Find_next(i / 2) +
     1) {
    for (auto j = (int64_t)i * i / 2; j < bits.size(); j += i)
      bits[j] = 0;
  }
}

//count all the divisors of a number
int divCount(int n) {
  int total = 1;
  int count = 0;
```

```
  int p = 2;
  if (n % p == 0) {
    while (n % p == 0) {
      n = n / p;
      count++;
    }
    total = total * (count + 1);
  }
  for (p = 3; p <= n; p += 2) {
    if (bits[p / 2]) {
      count = 0;
      if (n % p == 0) {
        while (n % p == 0) {
          n = n / p;
          count++;
        }
        total = total * (count + 1);
      }
    }
  }
  return total;
}
```

4.5 Prime Factorization (Log N)

```
int NMAX = 1e6;
vector<int> spf(MAXN + 1, 1);

void sieve() {
  spf[0] = 0;
  for (int i = 2; i <= NMAX; i++) {
    if (spf[i] == 1) {
      for (int j = i; j <= NMAX; j += i) {
        if (spf[j] == 1)
          spf[j] = i;
      }
    }
  }
}

void getFactorization(int x, map<int, int> &primes) {
  while (x != 1) {
    primes[spf[x]]++;
    x = x / spf[x];
  }
}
```

4.6 Calculate All Divisors

```
void divs() {
    const int n = 1e6;
    vector<vector<int>> divs(n);

    for (int i = 1; i < n; i++) {
        for (int j = i; j < n; j += i) {
            divs[j].push_back(i);
        }
    }
}

void div(int n) {
    vector<int> divisors;
    for (int i = 1; i*i <= n; ++i) {
        if (n % i == 0) {
```

```
            divisors.push_back(i);
            if (i != n / i) {
                divisors.push_back(n / i);
            }
        }
    }
}
```

4.7 Prime Factorization

```
{
  map<int, int> primes;
  for (int i = 2; i * i <= n; i++) {
    while (n % i == 0) {
      primes[i]++; // i is a prime
      n /= i;
    }
  }

  if (n != 1) {
    primes[n]++;
  }

  for (auto [a, b] : primes)
    cout << a << endl;
}
```

4.8 Fast matrix mult and pow

```
ll n = 3, MOD = 1e9 + 7; // change n and MOD to meet your requirements

vvi matMul(vvi &a, vvi &b) {
    vvi c(n, vi(n, 0));
    int k;
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            for (k = 0; k < n; k++) {
                c[i][j] += (a[i][k] % MOD) * (b[k][j] % MOD);
                c[i][j] %= MOD;
            }
    return c;
}

vvi matPow(vvi base, ll p) {
    vvi ans(n, vi(n, 0)); int i, j;
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
            ans[i][j] = (i == j);
    while (p) {
        if (p & 1)
            ans = matMul(ans, base);
        base = matMul(base, base);
        p >>= 1;
    }
    return ans;
}

[[1 , 1] , [1 , 0]] ** n = [[fib(n+1) , fib(n)], [fib(n) , fib(n-1)]]
// FIBONACCI FORMULA
```

4.9 Josephus

```
// Complete Search:
```

```
// Use list<int> or vector<int> to simulate the process

// Special Case k = 2 (skipping rule = 2)
// if n = 1b1b2b3..bn (in binary format) then the surviver is b1b2b3..bn1

// General Case:
// n = number of men
// k = skipping step
// Output: 0-based index of the surviver (add 1 if you want it to become 1-based)
int josephus(int n, int k) {
  if (n == 1)
    return 0;
  return (josephus(n - 1, k) + k) % n;
}
```

4.10 Pascal

```

    vector<vector<Z>> pascal;
    void build_pascal(int d) {
        pascal = {{1}};
        while (d--) {
            vector<Z> &lastrow = pascal.back();
            int s = lastrow.size();
            vector<Z> newrow(s + 1);
            newrow.front() = 1;
            newrow.back() = 1;
            for (int i = 1; i < s; i++)
                newrow[i] = lastrow[i] + lastrow[i - 1];
            pascal.push_back(newrow);
        }
    }
```

4.11 Modular Inverse

```
// a and b must be co-prime. returns (1 / a) mod b.
ll mod_inv(ll a, ll b) {
    return 1 < a ? b - mod_inv(b % a, a) * b / a : 1;
}
```

5 Miscellaneous

5.1 Gcd & Lcm

```
i64 gcd(i64 a, i64 b) { // binary GCD uses about 60% fewer bit operations
  if (!a) return b;

  u64 shift = __builtin_ctzll(a | b);
  a >>= __builtin_ctzll(a);

  while (b) {
    b >>= __builtin_ctzll(b);

    if (a > b)
      swap(a, b);
    b -= a;
  }
  return a << shift;
}
```

```

i64 lcm(i64 a, i64 b) {
  return a / gcd(a, b) * b;
}
```

5.2 Debugging tools

```
#define  rforeach(_it, c)     for(__typeof((c).rbegin()) _it = (c).rbegin(); _it != (c).rend(); ++_it)
#define  foreach(_it, c)      for(__typeof((c).begin()) _it = (c).begin(); _it != (c).end(); ++_it)
#define  all(a)               (a).begin(), (a).end()
#define  sz(a)                (int)a.size()
#define  endl                 '\n'

typedef int64_t  ll;

template <typename F, typename S>
ostream & operator << (ostream & os, const pair <F, S> & p)
{ return os << "(" << p.first << ", " << p.second << ")"; }

template <typename F, typename S>
ostream & operator << (ostream & os, const map <F, S> & _mp)
{ os << "["; foreach(it, _mp) { if(it != _mp.begin()) os << ", "; os << it->first << " = " << it->second; } return os << "]"; }

template <typename T>
ostream & operator << (ostream & os, const vector <T> & _v)
{ os << "["; foreach(it, _v) { if(it != _v.begin()) os << ", "; os << *it; } return os << "]"; }

template <typename T>
ostream & operator << (ostream & os, const set <T> & _st)
{ os << "["; foreach(it, _st) { if(it != _st.begin() ) os << ", "; os << *it; } return os << "]"; }

template <typename T, size_t S>
ostream & operator << (ostream & os, const array <T, S> & _ar)
{ os << "["; foreach(it, _ar) { if(it != _ar.begin() ) os << ", "; os << *it; } return os << "]"; }

template <typename T> void write(T _begin, T _end)
{ for(auto i = _begin; i != _end; ++i) cout << (*i) << ' '; cout << endl; }

template <typename T> void read(T _begin, T _end)
{ for(auto i = _begin; i != _end; ++i) cin >> (*i); }
```

5.3 Pseudo random number generator

```
/** pseudo-random number generator | C++xx >= C++11 **/

mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());

T myRand(T a, T b) {
  return uniform_int_distribution <T> (a, b)(rng);
}
```

5.4 Increase the stack memory limit

```
    static void run_with_stack_size(void (*func)(void), size_t stsize) {
        char *stack, *send;
        stack = (char *)malloc(stsize);
```

```
        send = stack + stsize - 16;
        send = (char *)((uintptr_t)send / 16 * 16);
        asm volatile(
            "mov %%rsp, (%0)\n"
            "mov %0, %%rsp\n"
            :
            : "r"(send));
        func();
        asm volatile("mov (%0), %%rsp\n" : : "r"(send));
        free(stack);
    }

    int32_t main() {
        run_with_stack_size(main_, 1024 * 1024 * 1024 / 2); // run with a 512 MB stack
        return 0;
    }
```

5.5 Stress test

```
g++ -o A A.cpp
g++ -o B B.cpp
g++ -o gen gen.cpp
for ((i = 1; ; ++i)); do  # if they are same then will loop forever
    echo $i
    ./gen $i > int
    ./A < int > out1
    ./B < int > out2
    #diff -w out1 out2 || break
     diff -w <(./A < int) <(./B < int) || break
done
```

5.6 Extra stuff

- #pragma GCC optimize ("Ofast") will make GCC auto-vectorize for loops and optimizes floating points better (assumes associativity and turns off denor
- #pragma GCC target ("avx,avx2") can double performance of vectorized code, but causes crashes on old machines. Also consider older #pragma GCC target ("sse4").
- #pragma GCC optimize ("trapv") kills the program on integer overflows (but is really slow).

Binomial coefficients have many different properties.

- Symmetry rule:

$$\binom{n}{k} = \binom{n}{n-k}$$

- Factoring in:

$$\binom{n}{k} = \frac{n}{k}\binom{n-1}{k-1}$$

- Sum over k:

$$\sum_{k=0}^{n} \binom{n}{k} = 2^n$$

- Sum over n:

$$\sum_{m=0}^{n} \binom{m}{k} = \binom{n+1}{k+1}$$

- Sum over n and k:

$$\sum_{k=0}^{m} \binom{n+k}{k} = \binom{n+m+1}{m}$$

- Sum of the squares:

$$\binom{n}{0}^2 + \binom{n}{1}^2 + \cdots + \binom{n}{n}^2 = \binom{2n}{n}$$

- Weighted sum:

$$1\binom{n}{1} + 2\binom{n}{2} + \cdots + n\binom{n}{n} = n2^{n-1}$$

Arithmetic Progression
Sequence:

$$a, a+d, a+2d, \ldots, a+(n-1)d, \ldots$$

Common Difference:

$$\texttt{Common difference} = d = a_2 - a_1$$

General Term (nth Term):

$$a_n = a + (n-1)d$$

nth Term from the Last Term:

$$a_n = l - (n-1)d$$

Sum of First n Terms:

$$S_n = \sum_{k=1}^{n} (a + (k-1)d) = \frac{n}{2}(2a + (n-1)d)$$

Geometric Progression
Sequence:

$$a, ar, ar^2, \ldots, ar^{n-1}, \ldots$$

Common Ratio:

$$\texttt{Common ratio} = r = \frac{a_{n+1}}{a_n}$$

General Term (nth Term):

$$a_n = ar^{n-1}$$

nth Term from the Last Term:

$$a_n = \frac{l}{r^{n-1}}$$

Sum of First n Terms:

$$S_n = \sum_{k=1}^{n} ar^{k-1} = \begin{cases} \frac{a(1-r^n)}{1-r} & \texttt{if } |r| < 1 \\ \frac{a(r^n-1)}{r-1} & \texttt{if } |r| > 1 \end{cases}$$

Harmonic Progression
Sequence:

$$\frac{1}{a}, \frac{1}{a+d}, \frac{1}{a+2d}, \ldots, \frac{1}{a+(n-1)d}, \ldots$$

General Term (nth Term):

$$a_n = \frac{1}{\frac{1}{a} + (n-1)\frac{1}{d}}$$

Sum of First n Terms:

$$S_n = \sum_{k=1}^{n} \frac{1}{a+(k-1)d} = n\left(\frac{1}{a} + \frac{1}{a+d} + \cdots + \frac{1}{a+(n-1)d}\right)$$

Power Series

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}, \qquad \sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6},$$
$$\sum_{i=1}^{n} i^3 = \left(\frac{n(n+1)}{2}\right)^2.$$

In general:

$$\sum_{i=1}^{n} i^m = \frac{1}{m+1}\left[(n+1)^{m+1} - 1 - \sum_{i=1}^{n}\left((i+1)^{m+1} - i^{m+1} - (m+1)i^m\right)\right]$$
$$\sum_{i=1}^{n-1} i^m = \frac{1}{m+1}\sum_{k=0}^{m}\binom{m+1}{k} B_k n^{m+1-k}.$$

Geometric Series

$$\sum_{i=0}^{n} c^i = \frac{c^{n+1}-1}{c-1}, \quad c \neq 1,$$
$$\sum_{i=0}^{\infty} c^i = \frac{1}{1-c}, \qquad |c| < 1,$$
$$\sum_{i=1}^{\infty} c^i = \frac{c}{1-c}, \qquad |c| < 1.$$

$$\sum_{i=0}^{n} ic^i = \frac{nc^{n+2} - (n+1)c^{n+1} + c}{(c-1)^2}, \quad c \neq 1,$$
$$\sum_{i=0}^{\infty} ic^i = \frac{c}{(1-c)^2}, \qquad |c| < 1.$$

Harmonic Series

$$H_n = \sum_{i=1}^{n} \frac{1}{i},$$
$$\sum_{i=1}^{n} iH_i = \frac{n(n+1)}{2}H_n - \frac{n(n-1)}{4}.$$

$$\sum_{i=1}^{n} H_i = (n+1)H_n - n,$$
$$\sum_{i=1}^{n}\binom{i}{m} H_i = \binom{n+1}{m+1}\left(H_{n+1} - \frac{1}{m+1}\right).$$

5.7 Knapsack dp

```
{
  // knapsack based on the weights with 1d dp
for (int i = 1; i <= n; i++)
  for (int j = W; j >= w[i]; j--)
    f[j] = max(f[j], f[j - w[i]] + v[i]);
}
```